PHY 3902 Experiment 3 Lab Report

Author(s): Michelle Johnstone, Patrick Geraghty
Affiliation: University of Ottawa
Date: October 15, 2025

Abstract

This experiment analyzed a 650nm laser diode with a built-in series resistor and lens with 5 volts of power applied by shining it on a light-dependent resistor (LDR) used to analyze the intensity of the laser's light. The setup was used for three similar experiments to determine the polarization of the beam as well as observe the effects of a linearly polarized filter in the beam path at various angles. The next step was verifying Brewster's angle by rotating a glass slide in the beam path and analyzing the relationship between the angle of incidence into the slide and the light intensity. The final step was achieving a Gaussian bell curve for the shape and intensity distribution of a beam using a blade to slowly uncover the beam and graphing the gradient of the intensity readings as the blade moved. All the results were as expected; it was determined that the laser's beam was partially linearly polarized and the graph of the light intensity read by the LDR as a function of the angle was sinusoidal with minima when the filter was orthogonal to the direction of the partial polarization of the beam. Brewster's angle was calculated to be 56°, which is approximately the theoretical value of 56.3°, and the intensity distribution bell curve was fairly close to the expected shape with a slight shift to one side.

1 Introduction

A linear polarizer limits an electric field incident on it to just one plane. When a beam that is already linearly polarized is incident on a second linearly polarizing filter that is in the same orientation as the first, the transmitted beam will have the same power after both filters. However, if the second filter is placed at an angle θ relative to the first, the transmitted beam will have intensity I calculated with the equation:

$$I = I_0 \cos^2(\theta) \tag{1}$$

This relationship is evident when plotting the intensity as a function of the angle, as the maxima and minima range from 0 to I sinusoidally.

Brewster's angle is the angle of incidence at which, when unpolarized light hits a barrier between two media, the reflected ray is perfectly polarized and the transmitted ray is partially polarized. When p-polarized light is incident on a barrier, the light is entirely transmitted; none of it is reflected. This is due to a phenomenon where the only light reflected from the medium is s-polarized because the reflected and refracted rays are 90° apart. At Brewster's angle, the electric field of the p-polarized light, which oscillates perpendicular to the wave's direction of propagation, would need to oscillate in the direction of the transmitted ray, whereas the electric field of the s-polarized light oscillates perpendicular to the angle of incidence and is reflected. Thus, all of the light is focused into the medium when the incident wave is p-polarized. The determination of Brewster's angle θ_B is dependent entirely on the indices of refraction of the two media

on either side of the barrier. The theoretical value for the expected angle of incidence at which there is no reflected light can be easily calculated using:

$$\theta_B = \arctan\left(\frac{n_2}{n_1}\right) \tag{2}$$

where n_1 is the index of refraction of the first medium and n_2 the index of refraction of the medium the laser beam is propagating into.

For the final portion wherein the Gaussian properties of the laser beam were investigated, a sharp edge is placed on a translational stage in front of a beam and is moved perpendicular to the beam's axis to slowly block more of the beam. As more of the beam is covered, the plot of the intensity distribution is an error function:

$$erf(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt \tag{3}$$

for which the derivative is the bell curve defined by:

$$f(x) = e^{-x^2} \tag{4}$$

This function dictates that the center of the beam is the brightest and the intensity radiates from there and diminishes proportionally to distance from the center.

2 Methods

For this experiment, the required equipment was few and inexpensive with a fairly simple set up. The laser diode was affixed to a 3D printed stand, and the LDR was set up at equivalent heights with enough separation to allow for the rotating slide to fit in between. The setup was stuck to the workbench using tack and tape in an attempt to stop the components from shifting. A linearly polarizing filter was then attached to a stepper motor on a stand, and the maximum intensity was recorded as the filter was rotated in increments of roughly 0.87° to get a good approximation of the direction of the laser beam's linear polarization to align the laser for the following steps. The first experiment was to analyze polarization by using a stepper motor to slowly turn a linearly polarizing filter in the beam path while measuring the resistance value of the LDR, which is inversely proportional to the intensity. The value of resistance was then graphed as a function of the angle of the filter relative to the starting point. Due to the relationship of the alignment being sinusoidal, using the change in angle of the filter's orientation is an acceptable way of graphing this relation.

The goal for the second section was to investigate Brewster's angle using a 3D printed fitted adaptor to attach a glass microscope slide to a stepper motor so the slide could be moved at increments of 0.18°. The motor was placed so the axis of rotation of the slide was centered on the beam path, so that the incident angle was most accurate and could be rotated 360° to attain 2 readings for Brewster's and achieve the highest accuracy.

The final portion of the experiment involved swapping the glass slide mounted on a motor for a knife-edge style 3D printout on a servo motor. This setup allowed for the beam to be covered and slowly be exposed to the LDR, then graph as a function of the degrees by which the servo was rotated, which is proportional to the amount of the beam uncovered. Using the LDR, it is possible to profile the laser beam, then the gradient of this distribution depicts the Gaussian properties of a beam of light.

3 Results and Discussion

For the first portion of the lab that involved checking the polarization of the beam, the results showed that the laser's beam was partially linearly polarized. The graph of the light intensity read by the LDR as a function of the angle was sinusoidal, with minima when the filter was orthogonal to the direction of the partial polarization of the beam. This is illustrated in Figure 1, which shows the measured intensity versus polarizer angle, demonstrating Malus's Law.

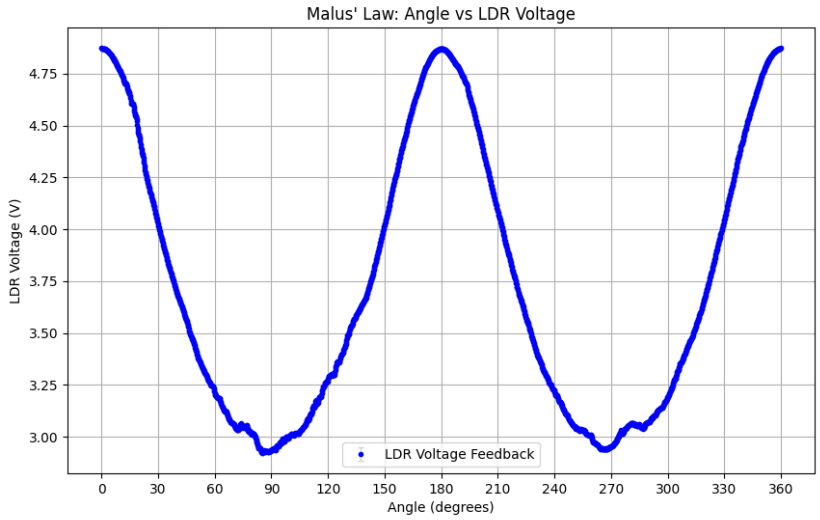


Figure 1: Measured intensity versus polarizer angle, demonstrating Malus's Law. Error bars represent measurement uncertainty. The solid line shows the best-fit cosine-squared curve.

Brewster's angle was calculated to be 56°, which is approximately the theoretical value of 56.3°. Figure 2 shows the measured intensity versus angle of incidence for p-polarized light, with the minimum indicating the Brewster angle, where reflected light is fully polarized.

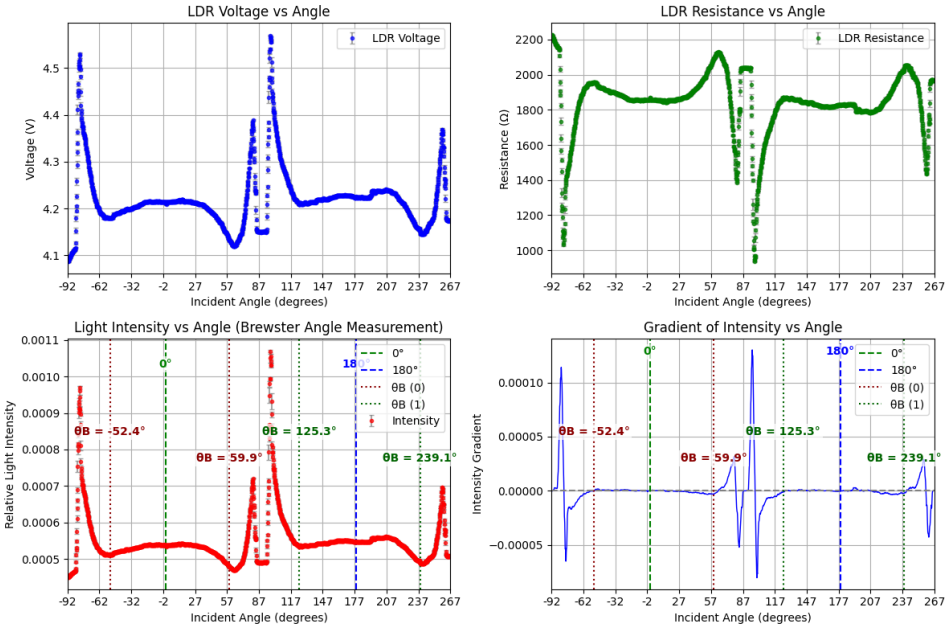


Figure 2: Measured intensity versus angle of incidence for p-polarized light. The minimum indicates the Brewster angle, where reflected light is fully polarized.

The intensity distribution bell curve was fairly close to the expected shape with a slight shift to one side. Figure 3 shows the measured intensity profile of the laser beam across its cross-section. The distribution is approximately Gaussian, indicating good beam quality.

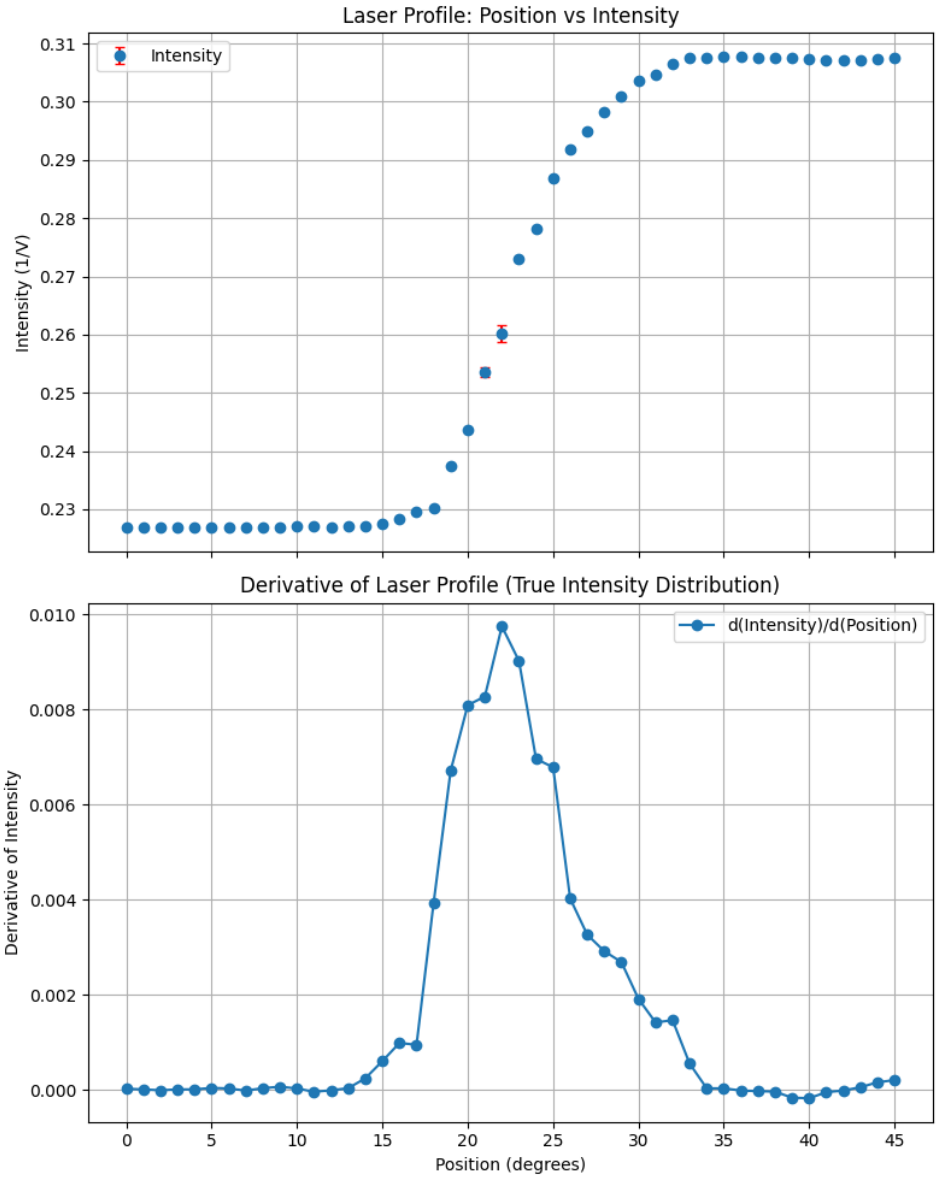


Figure 3: Measured intensity profile of the laser beam across its cross-section. The distribution is approximately Gaussian, indicating good beam quality.

Uncertainties were estimated based on the resolution of the stepper motor and the sensitivity of the LDR, with error sources including alignment, ambient light, and detector nonlinearity.

4 Conclusion

The experiment was able to adequately demonstrate the expected graph of the sinusoidal curve created by rotating a linearly polarized filter in front of a linearly polarized beam as well as Brewster's angle of incidence at which a beam polarized parallel to the plane of incidence is fully transmitted with no reflection. Brewster's angle for glass was expected to be 56.3° but was found to be roughly 56° with an error of 0.5

5 References

- Hecht, E. (2017). *Optics* (5th ed.). Pearson.
- Pedrotti, F. L., Pedrotti, L. S., and Pedrotti, L. M. (2017). *Introduction to Optics* (3rd ed.). Cambridge University Press.
- Experimental procedure and code: PHY 3902 course materials and scripts in experiment_3/src.
- Michel Godin; https://uottawa.brightspace.com/content/enforced/523665-2259FS0268421B00/PI (Accessed on September 30, 2025)
- Dr. Rüdiger Paschotta (RP); https://www.rp-photonics.com/brewster_s_angle.html (Accessed on October 12, 2025)
- Vinod Chugani; https://www.datacamp.com/tutorial/gaussian-distribution (Accessed on October 12, 2025)

6 Supplementary Code

Python Scripts

```python
import numpy as np
import os
import pandas as pd
import matplotlib.pyplot as plt

# Define the path to the data files
data_dir = "../data/malus"

CONSTANTS = {
    "V_SUPPLY": 5.0,  # Supply voltage in volts
    "R_FIXED": 9955,  # Fixed resistor value in ohms
    "STEPPER_STEPS_PER_REV": 2048,  # Steps per revolution for the stepper motor
    "ANALOG_RESOLUTION": 1023, # 10-bit ADC resolution
}

def plot_malus_law(data_file):
    # Load the data
    data_path = os.path.join(data_dir, data_file)
    data = pd.read_csv(data_path)

    # Extract angle and intensity data
    angles = 360 * data['step'] / CONSTANTS["STEPPER_STEPS_PER_REV"]  # Convert steps to degrees
    ldr_voltage_feedback = data['adc_mean'] * CONSTANTS["V_SUPPLY"] / CONSTANTS["ANALOG_RESOLUTION"]  # Convert ADC to voltage (0-5V range)
    ldr_voltage_stddev = data['adc_stddev'] * CONSTANTS["V_SUPPLY"] / CONSTANTS["ANALOG_RESOLUTION"]  # Convert ADC stddev to voltage stddev

    # Find index of first maximum
```

```
    first_max_idx = np.argmax(ldr_voltage_feedback)

    # Roll arrays so first maximum is at index 0
    angles = np.roll(angles, -first_max_idx)
    ldr_voltage_feedback = np.roll(ldr_voltage_feedback, -first_max_idx)
    ldr_voltage_stddev = np.roll(ldr_voltage_stddev, -first_max_idx)

    # Shift angles so that first maximum is at 0 and wrap around
    angles = angles - angles[0]
    angles = np.mod(angles, 360)

    # Plot the data
    plt.figure(figsize=(10, 6))
    plt.errorbar(angles, ldr_voltage_feedback, yerr=ldr_voltage_stddev,
        label='LDR Voltage Feedback', marker='o', linestyle='None',
        markersize=3, color='blue', ecolor='lightgray', elinewidth=1,
        capsize=2)
    plt.title("Malus' Law: Angle vs LDR Voltage")
    plt.xlabel('Angle (degrees)')
    plt.ylabel('LDR Voltage (V)')
    plt.xticks(np.arange(0, 361, 30))
    plt.grid()
    plt.legend()
    # Ensure output directory exists
    out_dir = os.path.abspath(os.path.join(os.path.dirname(__file__), '../out'))
    os.makedirs(out_dir, exist_ok=True)
    # Save plot to out directory
    out_path = os.path.join(out_dir, f"malus_plot_{os.path.splitext(data_file)[0]}.png")
    plt.savefig(out_path)
    plt.show()

def main():
    for file in os.listdir(data_dir):
        if file.endswith(".csv"):
            print(f"Processing file: {file}")
            plot_malus_law(file)

if __name__ == "__main__":
    main()
```

Listing 1: malus_plot.py

```
import os
from typing import List, Tuple, Optional, Dict, Any
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import sympy as sp
from error_propagation import propagate_error

# Experimental constants
CONSTANTS = {
    "V_SUPPLY": 5.0,            # Supply voltage (V)
    "R_FIXED": 9955,            # Fixed resistor value (ohms)
    "STEPPER_STEPS_PER_REV": 2048,  # Steps per full revolution for
```

```
        stepper motor
    "ANALOG_RESOLUTION": 1023   # 10-bit ADC resolution
}

# Path configuration
DATA_DIR = os.path.join(os.path.dirname(os.path.dirname(__file__)), "
   data", "brewster")

def group_consecutive_indices(
    indices: np.ndarray,
    max_gap: int = 3
) -> List[List[int]]:
    """
    Group indices that are within max_gap of each other.

    Args:
        indices: Array of indices to group
        max_gap: Maximum allowed gap between consecutive indices

    Returns:
        List of grouped indices
    """
    if len(indices) == 0:
        return []

    groups = []
    current_group = [indices[0]]

    for i in range(1, len(indices)):
        if indices[i] - indices[i-1] <= max_gap:
            current_group.append(indices[i])
        else:
            groups.append(current_group)
            current_group = [indices[i]]

    groups.append(current_group)
    return groups

def identify_perpendicular_angles(
    angles: pd.Series,
    intensities: pd.Series,
    window_size: int = 25
) -> List[Tuple[float, float, float, int, int]]:
    """
    Identify the flat regions between two steep slopes in the intensity
        data,
    which correspond to where the laser is perpendicular to the
       reflective surface.

    Note: Since the angles are relative to an arbitrary starting
       position,
    we're looking for the pattern in the data rather than specific
       angle values.

    Args:
        angles: Series of angles in degrees (relative to arbitrary
           start)
        intensities: Series of corresponding intensities
```

```
        window_size: Size of the moving window for smoothing

    Returns:
        perpendicular_regions: List of (start_angle, middle_angle,
            end_angle, start_idx, end_idx) tuples
            representing the regions where the laser is perpendicular
                to the surface
    """
    # Calculate and smooth the gradient
    gradient = np.gradient(intensities, angles)
    smoothed_gradient = np.convolve(gradient, np.ones(window_size)/
        window_size, mode='same')

    # Determine threshold for significant gradient changes (dynamic
        based on data)
    threshold = np.std(smoothed_gradient) * 1.5

    # Find indices where steep slopes (both positive and negative)
        occur
    pos_indices = np.where(smoothed_gradient > threshold)[0]
    neg_indices = np.where(smoothed_gradient < -threshold)[0]

    # Group consecutive indices and get the center of each group
    pos_groups = group_consecutive_indices(pos_indices)
    neg_groups = group_consecutive_indices(neg_indices)

    pos_centers = [int(np.mean(group)) for group in pos_groups if len(
        group) > 2]
    neg_centers = [int(np.mean(group)) for group in neg_groups if len(
        group) > 2]

    # Combine and sort all slope centers
    all_centers = sorted(pos_centers + neg_centers)

    # Find flat regions between consecutive steep slopes
    perpendicular_regions = []
    flat_region_threshold = threshold * 0.3  # Region is flat if
        average gradient < 30% of steep threshold
    min_region_width = 5  # Minimum width for a valid flat region

    for i in range(len(all_centers) - 1):
        start_idx = all_centers[i]
        end_idx = all_centers[i + 1]

        # Skip regions that are too narrow
        region_width = end_idx - start_idx
        if region_width < min_region_width:
            continue

        # Get angle values for this region
        middle_idx = (start_idx + end_idx) // 2
        start_angle = angles.iloc[start_idx]
        middle_angle = angles.iloc[middle_idx]
        end_angle = angles.iloc[end_idx]

        # Determine if region is flat by checking average gradient
            magnitude
        region_gradient_values = np.abs(smoothed_gradient[start_idx:
```

```
            end_idx])
        avg_gradient = np.mean(region_gradient_values)

        # Add region to results if it's flat enough
        if avg_gradient < flat_region_threshold:
            region_data = (
                start_angle,    # Start angle
                middle_angle,   # Middle angle
                end_angle,      # End angle
                start_idx,      # Start index
                end_idx         # End index
            )
            perpendicular_regions.append(region_data)

    return perpendicular_regions

def identify_brewster_angles(
    angles: pd.Series,
    intensities: pd.Series,
    perpendicular_regions: List[Tuple[float, float, float, int, int]],
    search_range_degrees: float = 60.0,
    smooth_window: int = 5
) -> List[Tuple[float, int, int]]:
    """
    Identify Brewster angles by finding the minima in reflection intensity
    on either side of the perpendicular incidence points.

    Args:
        angles: Series of angles in degrees
        intensities: Series of corresponding intensities
        perpendicular_regions: List of perpendicular region tuples (from identify_perpendicular_angles)
        search_range_degrees: Angular range to search around each perpendicular point (degrees)
        smooth_window: Window size for smoothing the intensity data

    Returns:
        brewster_angles: List of tuples (angle, index, region_id) representing identified Brewster angles
    """
    brewster_angles = []

    # Convert intensities to numpy array for processing
    intensities_array = intensities.to_numpy()

    # Apply light smoothing to reduce noise impact
    smoothed_intensities = np.convolve(intensities_array, np.ones(smooth_window)/smooth_window, mode='same')

    for region_id, (start_angle, middle_angle, end_angle, start_idx, end_idx) in enumerate(perpendicular_regions):
        # Define the search range around the perpendicular point
        lower_search_angle = middle_angle - search_range_degrees
        upper_search_angle = middle_angle + search_range_degrees

        # Find indices within the search range
        lower_indices = np.where((angles >= lower_search_angle) & (
```

```
            angles < middle_angle))[0]
        upper_indices = np.where((angles > middle_angle) & (angles <=
            upper_search_angle))[0]

        # Find minimum intensity in each range
        if len(lower_indices) > 0:
            # For the lower range (before perpendicular)
            lower_intensity_values = smoothed_intensities[lower_indices
                ]
            min_idx_in_lower = np.argmin(lower_intensity_values)
            min_idx_lower = lower_indices[min_idx_in_lower]
            brewster_angle_lower = angles.iloc[min_idx_lower]
            brewster_angles.append((brewster_angle_lower, min_idx_lower
                , region_id))

        if len(upper_indices) > 0:
            # For the upper range (after perpendicular)
            upper_intensity_values = smoothed_intensities[upper_indices
                ]
            min_idx_in_upper = np.argmin(upper_intensity_values)
            min_idx_upper = upper_indices[min_idx_in_upper]
            brewster_angle_upper = angles.iloc[min_idx_upper]
            brewster_angles.append((brewster_angle_upper, min_idx_upper
                , region_id))

    return brewster_angles

def setup_subplot(pos: int, x_data: pd.Series, y_data: pd.Series, title
    : str, ylabel: str, color: str = 'blue', yerr: Optional[pd.Series] =
     None, label: Optional[str] = None) -> Any:
    """Helper function to setup consistent subplot formatting"""
    ax = plt.subplot(2, 2, pos)
    if yerr is not None:
        ax.errorbar(x_data, y_data, yerr=yerr, fmt='o', markersize=3,
            color=color, ecolor='gray', alpha=0.7, capsize=2, label=
            label)
    else:
        ax.scatter(x_data, y_data, marker='o', s=3, color=color, label=
            label)
    if label:
        ax.legend()
    ax.set_title(title)
    ax.set_xlabel('Incident␣Angle␣(degrees)')
    ax.set_ylabel(ylabel)
    ax.set_xlim(0, 360)
    ax.set_xticks(np.arange(0, 361, 30))
    ax.grid(True)
    return ax

def relabel_x_axis(ax, offset: float):
    """
    Relabel the x-axis with incident angles while keeping data in place

    Args:
        ax: Matplotlib axis to relabel
        offset: Angle offset for 0   reference point
    """
    # Current ticks (evenly spaced from 0 to 360)
```

```
    original_ticks = np.arange(0, 361, 30)

    # Create new tick labels that are shifted by the offset
    # First, convert to the range -180 to 180
    new_labels = [(tick - offset) % 360 for tick in original_ticks]
    new_labels = [label - 360 if label > 180 else label for label in
        new_labels]

    # Find the minimum value
    min_label = min(new_labels)

    # Adjust labels to start from min_label and be strictly increasing
    adjusted_labels = []
    prev_label = min_label - 1  # Start below the minimum

    for label in new_labels:
        # If this label would decrease, add 360 to it
        while label <= prev_label:
            label += 360
        adjusted_labels.append(label)
        prev_label = label

    # Set the new tick labels without moving data points
    ax.set_xticks(original_ticks)
    ax.set_xticklabels([f"{int(label)}" for label in adjusted_labels])

    # Update xlabel to indicate the reference
    ax.set_xlabel('Incident␣Angle␣(degrees)')

def get_incident_angle_label(region_index: int) -> str:
    """
    Get a label for a perpendicular region.

    Args:
        region_index: The index of the region

    Returns:
        A string label (0   for first, 180   for second, etc.)
    """
    return f"{region_index␣*␣180}  "

def highlight_regions(ax, regions: List[Tuple], offset: float = 0.0,
    alpha: float = 0.3, show_region: bool = False):
    """
    Highlight perpendicular regions on a plot

    Args:
        ax: Matplotlib axis to draw on
        regions: List of perpendicular region tuples
        offset: Angle offset for 0   reference point
        alpha: Transparency for region highlighting
        show_region: Whether to show the span of the region
    """
    # Define distinct colors for each perpendicular angle line
    colors = ['green', 'blue', 'purple', 'orange', 'cyan', 'magenta']

    # For each region, highlight and label with the appropriate angle
    for i, (start_angle, middle_angle, end_angle, _, _) in enumerate(
```

```
        regions):
268         # Calculate the label text (0   for first, 180   for second,
                etc.)
269         label_text = get_incident_angle_label(i)
270
271         # Get color for this line (cycle through colors if more regions
                than colors)
272         color = colors[i % len(colors)]
273
274         # Highlight the middle angle - this is our best guess for the
                perpendicular angle
275         line = ax.axvline(x=middle_angle, color=color, linestyle='--',
                label=label_text)
276
277         # Add text label near the line at the top of the plot
278         y_pos = ax.get_ylim()[1] * 0.95  # Position at 95% of the y-
                axis height
279         ax.text(middle_angle, y_pos, label_text,
280                 color=color, fontweight='bold', ha='center', va='top',
281                 bbox=dict(facecolor='white', alpha=0.7, edgecolor='none
                    ', pad=2))
282
283         # Show the region bounds if requested
284         if show_region:
285             # Use a lighter version of the same color for the region
286             region_color = color
287             ax.axvspan(start_angle, end_angle, color=region_color,
                    alpha=alpha)
288
289     # Add a legend only if regions were found
290     if regions:
291         handles, labels = ax.get_legend_handles_labels()
292         # Filter to only include the perpendicular angle lines we just
                added
293         perpendicular_handles = [h for h, l in zip(handles, labels) if
                '  ' in l]
294         perpendicular_labels = [l for l in labels if '  ' in l]
295         if perpendicular_handles:
296             ax.legend(perpendicular_handles, perpendicular_labels, loc=
                    'best')
297
298 def highlight_brewster_angles(ax, brewster_angles: List[Tuple],
        angle_offset: float = 0.0):
299     """
300     Highlights Brewster angles on the plot with vertical lines
301
302     Args:
303         ax: Matplotlib axis to draw on
304         brewster_angles: List of tuples (angle, index, region_id)
305         angle_offset: Angle offset for 0   reference point
306     """
307     if not brewster_angles:
308         return
309
310     # Colors for Brewster angle lines - use a different color scheme
            than perpendicular lines
311     brewster_colors = ['darkred', 'darkgreen', 'darkblue', 'darkorange'
            , 'darkmagenta', 'darkcyan']
```

```
312
313     # Group brewster angles by their associated perpendicular region
314     brewster_by_region = {}
315     for ba_angle, _, region_id in brewster_angles:
316         if region_id not in brewster_by_region:
317             brewster_by_region[region_id] = []
318         brewster_by_region[region_id].append(ba_angle)
319
320     # Draw each Brewster angle with its label
321     for region_id, angles_list in brewster_by_region.items():
322         # Use a color based on the region ID
323         color = brewster_colors[region_id % len(brewster_colors)]
324
325         # For each angle in this region group
326         for i, angle in enumerate(angles_list):
327             # Get the incident angle for labeling
328             incident_angle = angle - angle_offset
329
330             # Create a label that shows the incident angle
331             label = f" B ␣=␣{incident_angle:.1f}  "
332
333             # Add a vertical line for this Brewster angle
334             line = ax.axvline(x=angle, color=color, linestyle=':',
                    linewidth=1.5,
335                               label=f" B ␣({region_id})" if i==0 else ""
                                  )
336
337             # Calculate text position - alternate between top and
                bottom for visibility
338             if i % 2 == 0:
339                 y_frac = 0.65  # 65% up the axis
340                 va = 'top'
341             else:
342                 y_frac = 0.50  # 50% up the axis
343                 va = 'bottom'
344
345             # Add text label at the appropriate position using axis
                transform
346             ax.text(angle, y_frac, label,
347                     color=color, fontweight='bold', ha='center', va=va,
348                     bbox=dict(facecolor='white', alpha=0.7, edgecolor='
                        none', pad=2),
349                     transform=ax.get_xaxis_transform(),
350                     )
351
352     # Update legend with Brewster angle entries
353     handles, labels = ax.get_legend_handles_labels()
354     ax.legend(handles, labels, loc='best')
355
356 def process_angle_data(data_file: str) -> tuple[pd.Series, pd.Series,
    pd.Series, pd.Series, pd.Series, pd.Series, pd.Series]:
357     """
358     Process raw angle data from a CSV file into angles and intensities
359
360     Args:
361         data_file: Path to the CSV data file
362
363     Returns:
```

```
        Tuple of (angles, ldr_voltage_feedback, ldr_resistance,
            intensities, adc_stddev)
    """
    # Load the data
    data = pd.read_csv(data_file)

    # Extract angle and intensity
    angles = 360 * data['step'] / CONSTANTS["STEPPER_STEPS_PER_REV"]  #
        Convert steps to degrees

    # Convert LDR voltage to light intensity
    ldr_voltage_feedback = data['adc_mean'] * CONSTANTS["V_SUPPLY"] /
        CONSTANTS["ANALOG_RESOLUTION"]  # Convert ADC to voltage (0-5V
        range)

    # Calculate LDR resistance from voltage divider
    ldr_resistance = CONSTANTS["R_FIXED"] * (CONSTANTS["V_SUPPLY"] -
        ldr_voltage_feedback) / ldr_voltage_feedback

    # Convert resistance to relative light intensity (I     1/R)
    intensities = 1.0 / ldr_resistance

    # Extract adc_stddev
    adc_stddev = data['adc_stddev']

    # Error propagation for resistance
    V = sp.Symbol('V')
    Rf = CONSTANTS["R_FIXED"]
    Vs = CONSTANTS["V_SUPPLY"]
    R_expr = Rf * (Vs - V) / V
    resistance_uncertainty = []
    for v, dv in zip(ldr_voltage_feedback, adc_stddev * Vs / CONSTANTS[
        "ANALOG_RESOLUTION"]):
        resistance_uncertainty.append(propagate_error(R_expr, [V], {V:
            v}, {V: dv}))
    resistance_uncertainty = pd.Series(resistance_uncertainty)

    # Error propagation for intensity
    I = sp.Symbol('I')
    intensity_expr = 1 / R_expr
    intensity_uncertainty = []
    for v, dv in zip(ldr_voltage_feedback, adc_stddev * Vs / CONSTANTS[
        "ANALOG_RESOLUTION"]):
        intensity_uncertainty.append(propagate_error(intensity_expr, [V
            ], {V: v}, {V: dv}))
    intensity_uncertainty = pd.Series(intensity_uncertainty)

    return angles, ldr_voltage_feedback, ldr_resistance, intensities,
        adc_stddev, resistance_uncertainty, intensity_uncertainty

def create_brewster_plots(
    angles: pd.Series,
    ldr_voltage_feedback: pd.Series,
    ldr_resistance: pd.Series,
    intensities: pd.Series,
    perpendicular_regions: List[Tuple],
    brewster_angles: List[Tuple],
    adc_stddev: pd.Series,
```

```
    resistance_uncertainty: pd.Series,
    intensity_uncertainty: pd.Series,
) -> None:
    """
    Create visualization plots for Brewster angle analysis, now with error bars
    """
    fig = plt.figure(figsize=(12, 8))

    # Determine the angle offset based on perpendicular regions
    angle_offset = 0
    if perpendicular_regions:
        angle_offset = perpendicular_regions[0][1]  # Middle angle of first region

    # Plot 1: Raw voltage (with error bars)
    voltage_ax = setup_subplot(
        1, angles,
        ldr_voltage_feedback,
        'LDR Voltage vs Angle',
        'Voltage (V)',
        'blue', yerr=adc_stddev * CONSTANTS["V_SUPPLY"] / CONSTANTS["ANALOG_RESOLUTION"],
        label='LDR Voltage'
    )
    relabel_x_axis(voltage_ax, angle_offset)

    # Plot 2: Calculated resistance (with error bars)
    resistance_ax = setup_subplot(
        2,
        angles, ldr_resistance,
        'LDR Resistance vs Angle',
        'Resistance (  )',
        'green', yerr=resistance_uncertainty,
        label='LDR Resistance'
    )
    relabel_x_axis(resistance_ax, angle_offset)

    # Plot 3: Light intensity with perpendicular regions (with error bars)
    intensity_ax = setup_subplot(
        3,
        angles,
        intensities,
        'Light Intensity vs Angle (Brewster Angle Measurement)',
        'Relative Light Intensity',
        'red',
        yerr=intensity_uncertainty,
        label='Intensity'
    )
    relabel_x_axis(intensity_ax, angle_offset)

    # Calculate gradient for visualization
    gradient = np.gradient(intensities, angles)
    smoothed_gradient = np.convolve(gradient, np.ones(25)/25, mode='same')

    # Plot 4: Gradient visualization
```

```
    gradient_ax = plt.subplot(2, 2, 4)
    gradient_ax.plot(angles, smoothed_gradient, color='blue', linewidth=1)
    gradient_ax.set_title('Gradient of Intensity vs Angle')
    gradient_ax.set_ylabel('Intensity Gradient')
    gradient_ax.set_xlim(0, 360)
    gradient_ax.set_xticks(np.arange(0, 361, 30))
    gradient_ax.grid(True)
    gradient_ax.axhline(y=0, color='gray', linestyle='--')
    relabel_x_axis(gradient_ax, angle_offset)

    # Highlight regions and Brewster angles on intensity and gradient plots
    highlight_regions(intensity_ax, perpendicular_regions, offset=angle_offset)
    highlight_brewster_angles(intensity_ax, brewster_angles, angle_offset)
    highlight_regions(gradient_ax, perpendicular_regions, offset=angle_offset)
    highlight_brewster_angles(gradient_ax, brewster_angles, angle_offset)

    plt.tight_layout()
    # Ensure output directory exists
    out_dir = os.path.abspath(os.path.join(os.path.dirname(__file__), '../out'))
    os.makedirs(out_dir, exist_ok=True)
    # Save plot to out directory
    out_path = os.path.join(out_dir, 'brewster_plot.png')
    plt.savefig(out_path)
    plt.show()

def plot_brewster_angle(data_file: str, angle_uncertainty: float = 1.0):
    """
    Process and plot Brewster angle data from a CSV file.
    """
    # Load and process the data
    data_path = os.path.join(DATA_DIR, data_file)
    angles, ldr_voltage_feedback, ldr_resistance, intensities, adc_stddev, resistance_uncertainty, intensity_uncertainty = process_angle_data(data_path)

    # Identify perpendicular angles (where laser is perpendicular to surface)
    perpendicular_regions = identify_perpendicular_angles(angles, intensities)

    # Identify Brewster angles based on intensity minima around perpendicular points
    brewster_angles = identify_brewster_angles(angles, intensities, perpendicular_regions)

    # Create the plots
    create_brewster_plots(angles, ldr_voltage_feedback, ldr_resistance, intensities,
                          perpendicular_regions, brewster_angles, adc_stddev,
```

```
                        resistance_uncertainty, intensity_uncertainty,
                        )

    return angles, intensities, perpendicular_regions, brewster_angles

def report_perpendicular_regions(
    perpendicular_regions: List[Tuple],
    angles: pd.Series,
    intensities: pd.Series
) -> None:
    """
    Print detailed analysis of the identified perpendicular regions.

    Args:
        perpendicular_regions: List of identified perpendicular regions
        angles: Series of angle measurements
        intensities: Series of intensity measurements
    """
    # Calculate the angle offset based on the first perpendicular region
    angle_offset = perpendicular_regions[0][1] if perpendicular_regions else 0

    for i, (start_angle, middle_angle, end_angle, start_idx, end_idx) in enumerate(perpendicular_regions):
        # Calculate the width of the region in degrees
        region_width = end_angle - start_angle

        # Calculate the flatness metric (lower is flatter)
        region_gradient = np.gradient(intensities.iloc[start_idx:end_idx], angles.iloc[start_idx:end_idx])
        flatness = np.mean(np.abs(region_gradient))

        # Calculate incident angles
        middle_incident = middle_angle - angle_offset
        start_incident = start_angle - angle_offset
        end_incident = end_angle - angle_offset

        print(f"␣␣Region␣{i+1}:")
        print(f"␣␣␣␣Raw␣angle␣center:␣{middle_angle:.1f}  ")
        print(f"␣␣␣␣Incident␣angle␣center:␣{middle_incident:.1f}  ")
        print(f"␣␣␣␣Raw␣spans:␣{start_angle:.1f}  ␣to␣{end_angle:.1f}␣(width:␣{region_width:.1f}  )")
        print(f"␣␣␣␣Incident␣spans:␣{start_incident:.1f}  ␣to␣{end_incident:.1f}  ")
        print(f"␣␣␣␣Flatness␣metric:␣{flatness:.5f}")

        # Map to theoretical angles - assumes the first region is 0 and second is 180
        if len(perpendicular_regions) >= 2:
            if i == 0:
                print("␣␣␣␣Maps␣to␣theoretical␣incident␣angle:␣0  ␣(laser␣perpendicular␣to␣surface)")
            elif i == 1:
                print("␣␣␣␣Maps␣to␣theoretical␣incident␣angle:␣180  ␣(laser␣perpendicular␣to␣surface)")

    # Calculate the relative angle between regions if we found both
```

```
    if len(perpendicular_regions) >= 2:
        angle_diff = perpendicular_regions[1][1] -
            perpendicular_regions[0][1]
        print(f"\n  Angle difference between perpendicular regions (raw
            ): {angle_diff:.1f}  ")
        print("  Expected difference: 180   (theoretical)")

        # Check if our angle difference is close to expected
        if abs(angle_diff - 180) < 30:
            print("     Angle difference is close to expected value
                (180  )")
        else:
            print("     Angle difference deviates significantly from
                expected value (180  )")
            print("     This may indicate issues with the measurement or
                detection algorithm.")

def report_brewster_angles(
    brewster_angles: List[Tuple],
    perpendicular_regions: List[Tuple],
    intensities: pd.Series,
    angle_offset: float
) -> None:
    """
    Print detailed analysis of the identified Brewster angles.

    Args:
        brewster_angles: List of brewster angle tuples (angle, index,
            region_id)
        perpendicular_regions: List of perpendicular region tuples
        intensities: Series of intensity measurements
        angle_offset: Angle offset for 0   reference
    """
    print("\nIdentified Brewster angles (reflection minima):")

    # Group the Brewster angles by their associated perpendicular
        region
    brewster_by_region = {}
    for angle, idx, region_id in brewster_angles:
        region_id = int(region_id)  # Ensure region_id is an integer
        if region_id not in brewster_by_region:
            brewster_by_region[region_id] = []
        brewster_by_region[region_id].append((angle, idx))

    for region_id, angles_list in brewster_by_region.items():
        perp_angle = perpendicular_regions[region_id][1]  # Middle
            angle of this region
        print(f"  Region {region_id} (centered at {perp_angle:.1f}  ):"
            )

        for i, (angle, idx) in enumerate(angles_list):
            incident_angle = angle - angle_offset
            intensity_value = intensities.iloc[idx]
            angle_diff = angle - perp_angle
            position = "before" if angle_diff < 0 else "after"

            print(f"    Brewster angle {i+1}: {angle:.1f}  raw, {
                incident_angle:.1f}  incident")
```

```
            print(f"    Position: {position} perpendicular point (offset {abs(angle_diff):.1f}  )")
            print(f"    Intensity value: {intensity_value:.8f}")

            # Theoretical Brewster angle is ~56   for glass with n=1.5
            if incident_angle > 90:
                incident_angle -= 180  # Normalize to -90   to +90
            print(f"    Experimental Brewster angle (incident): {incident_angle:.1f}  ")
            if 50 <= abs(incident_angle) <= 60:
                print(f"        Close to theoretical Brewster angle (~56.3  for glass with n=1.5)")
            else:
                print(f"        Deviates from expected Brewster angle range (45  -65  )")
            print("")

def main() -> None:
    """
    Main function to process and analyze all Brewster angle data files.
    """
    for file in os.listdir(DATA_DIR):
        if not file.endswith('.csv'):  # Only process CSV files
            continue

        print(f"\nAnalyzing data from {file}")

        # Check if this is p-polarized data (only these can show Brewster angle)
        is_p_polarized = "p_polarized" in file

        # Process data and generate plots
        if is_p_polarized:
            # For p-polarized data, identify both perpendicular regions and Brewster angles
            angles, intensities, perpendicular_regions, brewster_angles = plot_brewster_angle(file)
        else:
            # For other data (like s-polarized), don't look for Brewster angles
            # To maintain compatibility, we'll still use the same function but ignore the Brewster angles
            angles, intensities, perpendicular_regions, brewster_angles = plot_brewster_angle(file)
            # Clear any accidentally identified Brewster angles for non-p-polarized data
            brewster_angles = []

        # Find minimum intensity point
        min_idx = np.argmin(intensities)
        brewster_angle_raw = angles.iloc[min_idx]

        # Calculate the angle offset to get the incident angle
        angle_offset = 0
        if perpendicular_regions:
            angle_offset = perpendicular_regions[0][1]  # Middle angle of first perpendicular region

```

```
        # Calculate the minimum intensity angle in the incident angle reference frame
        brewster_incident_angle = brewster_angle_raw - angle_offset

        # For any data file, report the global minimum intensity
        print(f"Global minimum intensity (raw angle): {brewster_angle_raw:.1f}  ")
        print(f"Global minimum intensity (incident angle): {brewster_incident_angle:.1f}  ")

        # Report regions where the laser is likely perpendicular to the reflective surface
        print("Identified flat regions (where laser is perpendicular to surface):")

        if not perpendicular_regions:
            print("  No suitable perpendicular regions found. Try adjusting the detection parameters.")
        else:
            report_perpendicular_regions(perpendicular_regions, angles, intensities)

        # Only report on Brewster angles for p-polarized data
        if is_p_polarized and brewster_angles:
            report_brewster_angles(brewster_angles, perpendicular_regions, intensities, angle_offset)
        elif is_p_polarized and not brewster_angles:
            print("\nNo Brewster angles identified in this p-polarized data.")
            print("Try adjusting the search parameters or check data quality.")

if __name__ == "__main__":
    main()
```

Listing 2: brewster_plot.py

```
# Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import os

CONSTANTS = {
    "V_SUPPLY": 5.0,  # Supply voltage in volts
    "R_FIXED": 9955,  # Fixed resistor value in ohms, not used in current calculations
    "ANALOG_RESOLUTION": 1023, # 10-bit ADC resolution
}
DATA_PATH = '../data/laser_profile/laser_profile.csv'


def load_data(path):
    """Load CSV data and return DataFrame."""
    return pd.read_csv(path, index_col=False)


def calculate_intensity(df):
```

```
    """Calculate intensity and its standard deviation from LDR readings."""
    ldr_voltage = df['ldr_mean'] * CONSTANTS["V_SUPPLY"] / CONSTANTS["ANALOG_RESOLUTION"]
    ldr_voltage_stddev = df['ldr_stddev'] * CONSTANTS["V_SUPPLY"] / CONSTANTS["ANALOG_RESOLUTION"]
    intensity = 1 / ldr_voltage
    intensity_stddev = ldr_voltage_stddev / (ldr_voltage ** 2)
    result_df = pd.DataFrame({
        'position': df['position'],
        'intensity': intensity,
        'intensity_stddev': intensity_stddev
    })
    return result_df


def plot_laser_profile(intensity_df):
    """Plot intensity and its derivative in subplots."""
    # Sort by position
    sorted_df = intensity_df.sort_values('position')
    positions = sorted_df['position'].values
    intensities = sorted_df['intensity'].values
    intensities_stddev = sorted_df['intensity_stddev'].values

    # Calculate derivative
    intensity_derivative = np.gradient(intensities, positions)

    # Create subplots
    fig, axes = plt.subplots(2, 1, figsize=(8, 10), sharex=True)

    # Intensity plot
    axes[0].errorbar(positions, intensities, yerr=intensities_stddev, fmt='o', ecolor='red', capsize=3, label='Intensity')
    axes[0].set_ylabel('Intensity (1/V)')
    axes[0].set_title('Laser Profile: Position vs Intensity')
    axes[0].legend()
    axes[0].grid(True)

    # Derivative plot
    axes[1].plot(positions, intensity_derivative, marker='o', label='d(Intensity)/d(Position)')
    axes[1].set_xlabel('Position (degrees)')
    axes[1].set_ylabel('Derivative of Intensity')
    axes[1].set_title('Derivative of Laser Profile (True Intensity Distribution)')
    axes[1].legend()
    axes[1].grid(True)

    # Set x-ticks to increments of 5
    min_pos = positions.min()
    max_pos = positions.max()
    xticks = np.arange(min_pos, max_pos + 1, 5)
    axes[1].set_xticks(xticks)
    axes[0].set_xticks(xticks)

    plt.tight_layout()
    # Ensure output directory exists
    out_dir = os.path.abspath(os.path.join(os.path.dirname(__file__), '
```

```
        ../out'))
    os.makedirs(out_dir, exist_ok=True)
    # Save plot to out directory
    out_path = os.path.join(out_dir, 'laser_profile_plot.png')
    plt.savefig(out_path)
    plt.show()


def main():
    df = load_data(DATA_PATH)
    intensity_df = calculate_intensity(df)
    plot_laser_profile(intensity_df)


if __name__ == "__main__":
    main()
```

Listing 3: laser_profile$_plot.py$

```
import sympy as sp
import numpy as np

def propagate_error(R_expr, variables, values, uncertainties):
    """
    Propagate uncertainty for a function R using the general formula:
     R  = sqrt(   [ (  R  /  xi  )^2 * ( xi )^2 ])
    Args:
        R_expr: sympy expression for R (e.g., x*y)
        variables: list of sympy symbols (e.g., [x, y])
        values: dict mapping symbol to value (e.g., {x: 2, y: 3})
        uncertainties: dict mapping symbol to uncertainty (e.g., {x:
           0.1, y: 0.2})
    Returns:
        float: propagated uncertainty  R
    """
    total = 0.0
    for var in variables:
        partial = sp.diff(R_expr, var)
        partial_val = float(partial.evalf(subs=values))
        delta = uncertainties[var]
        total += (partial_val ** 2) * (delta ** 2)
    return np.sqrt(total)

# Example usage
def example():
    x, y = sp.symbols('x␣y')
    R = x * y
    values = {x: 2, y: 3}
    uncertainties = {x: 0.1, y: 0.2}
    delta_R = propagate_error(R, [x, y], values, uncertainties)
    print(f"Propagated␣uncertainty␣ R :␣{delta_R}")

if __name__ == "__main__":
    example()
```

Listing 4: error_propagation.py

Arduino Sketches

```

int const LASER_PIN = 3;
int const LDR_FEED_PIN = A0;
bool hasRun = false;

float averageReadLDR(int pin, int numSamples) {
    long total = 0;
    for (int i = 0; i < numSamples; i++) {
        total += analogRead(pin);
        delay(10); // Small delay to allow ADC to stabilize
    }
    return total / (float)numSamples;
}

void setup() {
    pinMode(LASER_PIN, OUTPUT);
    pinMode(LDR_FEED_PIN, INPUT);
    Serial.begin(9600);
}

void loop() {
    if (!hasRun) {
        digitalWrite(LASER_PIN, HIGH); // Turn the laser on

        float ldrValue = averageReadLDR(LDR_FEED_PIN, 100);
        Serial.print("Average␣LDR␣Value:␣");
        Serial.println(ldrValue);

        digitalWrite(LASER_PIN, LOW);  // Turn the laser off
        Serial.println("Laser␣deactivated.");
        hasRun = true;                 // Ensure this block runs only
           once
    }
}
```

Listing 5: laser_setup.ino

```
/*
 *  Laser Properties - stepper movement + voltage measurement
 *  This code is a basic sketch that highlights how to move a stepper
    motor and make a voltage measurement. In this experiment,
 *  you will have to modify this sketch to rotate the stepper motor in
    small increments and take a light intensity measurement at each
 *  increment. FOR loops are useful for actuating motors in small
    increments. Remember that you need to calibrate the stepper motor
    so that
 *  you know how many degrees you are rotating the stepper motor for a
    given number of "steps". Also, the light intensity measurement
    might
 *  be noisy, so calculating an average and uncertainties might be
    important!
 *
 *  - Michel Godin for PHY3902
 */

```

```
#include <Stepper.h>      //library required to use the stepper motor functions


// Define Variables
int laserPin = 3;     // We will use an output pin to power the laser.
const int ldrPin = A0; // LDR analog input pin
const int increment = 1; // Number of steps per increment (smaller for more data points)
const int numSamples = 100; // Number of LDR samples to average per position

// Stepper Motor constants
const int SPR = 32;
const int GEAR_RED = 64;
const int TOTAL_STEPS = SPR * GEAR_RED;  // Total number of steps for one full rotation of the stepper motor

// Create Instance of Stepper Class
// Specify Pins used for motor coils
// Pins are 8, 9, 10, 11
// Connected to ULN2003 Motor Driver IN1, IN2, IN3, IN4
// Pins entered in sequence 1-3-2-4 for proper stepper sequencing

Stepper stepperMotor(SPR, 8, 10, 9, 11);

/**
 * @brief Prints a value in CSV format to the Serial output.
 *
 * @tparam T The type of the value to print.
 * @param value The value to print.
 * @param eol If true, prints a newline after the value; otherwise, prints a comma.
 */
template <typename T>
void printSerialCSV(T value, bool eol = false, int precision = 4) {
  if (eol) {
    Serial.println((float)value, precision);
  } else {
    Serial.print((float)value, precision);
    Serial.print(",");
  }
}

/**
 * @brief Prints a full CSV row of the experiment data to Serial.
 * @param step The current step of the stepper motor.
 * @param adcValue The ADC value read from the sensor.
 * @param adcError The error associated with the ADC reading.
 */
void printDataCSVRow(
  int step, float adcMean, float adcStddev
) {
  printSerialCSV(step);
  printSerialCSV(adcMean);
  printSerialCSV(adcStddev, true);
}

```

```
/**
 * @brief Holds the mean and standard deviation of LDR readings.
 */
struct LDRStats {
  float mean;
  float stddev;
};

/**
 * @brief Averages multiple readings from an LDR connected to the specified analog pin.
 * @param pin The analog pin where the LDR is connected.
 * @param numSamples The number of samples to average.
 * @return LDRStats struct containing mean and standard deviation.
 */
LDRStats averageReadLDR(int pin, int numSamples) {
  float total = 0;
  float values[20]; // max 20 samples for memory safety
  if (numSamples > 20) numSamples = 20;
  for (int i = 0; i < numSamples; i++) {
    values[i] = analogRead(pin);
    total += values[i];
    delay(10); // Small delay to allow ADC to stabilize
  }
  float mean = total / (float)numSamples;
  float sumSq = 0;
  for (int i = 0; i < numSamples; i++) {
    sumSq += (values[i] - mean) * (values[i] - mean);
  }
  float stddev = sqrt(sumSq / (float)numSamples);
  LDRStats stats = {mean, stddev};
  return stats;
}

void setup() {
  pinMode(laserPin, OUTPUT);
  digitalWrite(laserPin, HIGH); // Turn on the laser
  stepperMotor.setSpeed(50);   // Slow speed for accuracy
  Serial.begin(9600);
  Serial.println("step,adc_mean,adc_stddev"); // CSV header
}

void loop() {
  static bool done = false;
  if (!done) {
    digitalWrite(laserPin, HIGH); // Laser ON
    for (int step = 0; step <= TOTAL_STEPS; step += increment) {
      stepperMotor.step(increment); // Move stepper
      LDRStats stats = averageReadLDR(ldrPin, numSamples); // Average LDR
      printDataCSVRow(step, stats.mean, stats.stddev); // Print CSV row
      delay(20); // Stability delay
    }
    digitalWrite(laserPin, LOW); // Laser OFF
    done = true;
  }
}
```

Listing 6: laser_properties_stepper.ino

```
#ifndef CSV_PRINTER_H
#define CSV_PRINTER_H

#include <Arduino.h>

/**
 * @brief A library for printing CSV formatted data to the Arduino Serial monitor.
 *
 * This library provides convenient functions for outputting data in CSV format,
 * making it easy to collect and analyze experimental data from Arduino projects.
 *
 * @author Generated for PHY3902 Labs
 * @version 1.0
 */

class CSVPrinter {
public:
    /**
     * @brief Initialize the CSV printer with Serial communication.
     * @param baudRate The baud rate for Serial communication (default: 9600)
     */
    static void begin(long baudRate = 9600);

    /**
     * @brief Prints a value in CSV format to the Serial output.
     * @tparam T The type of the value to print.
     * @param value The value to print.
     * @param eol If true, prints a newline after the value; otherwise, prints a comma.
     * @param precision Number of decimal places for floating point values (default: 4)
     */
    template <typename T>
    static void printValue(T value, bool eol = false, int precision = 4);

    /**
     * @brief Prints a CSV header row.
     * @param headers Array of header strings
     * @param numHeaders Number of headers in the array
     */
    static void printHeader(const char* headers[], int numHeaders);

    /**
     * @brief Prints a CSV header row using a single string with comma-separated values.
     * @param headerString String containing comma-separated header names
     */
    static void printHeader(const char* headerString);

```

```
47      /**
48       * @brief Starts a new CSV row (no output, just for organization)
49       */
50      static void startRow();
51
52      /**
53       * @brief Ends the current CSV row with a newline
54       */
55      static void endRow();
56
57      /**
58       * @brief Prints a complete CSV row with 2 values
59       */
60      template <typename T1, typename T2>
61      static void printRow(T1 val1, T2 val2, int precision = 4);
62
63      /**
64       * @brief Prints a complete CSV row with 3 values
65       */
66      template <typename T1, typename T2, typename T3>
67      static void printRow(T1 val1, T2 val2, T3 val3, int precision = 4);
68
69      /**
70       * @brief Prints a complete CSV row with 4 values
71       */
72      template <typename T1, typename T2, typename T3, typename T4>
73      static void printRow(T1 val1, T2 val2, T3 val3, T4 val4, int
            precision = 4);
74  };
75
76  // Template function implementations
77
78  template <typename T>
79  void CSVPrinter::printValue(T value, bool eol, int precision) {
80      if (eol) {
81          Serial.println((float)value, precision);
82      } else {
83          Serial.print((float)value, precision);
84          Serial.print(",");
85      }
86  }
87
88  template <typename T1, typename T2>
89  void CSVPrinter::printRow(T1 val1, T2 val2, int precision) {
90      printValue(val1, false, precision);
91      printValue(val2, true, precision);
92  }
93
94  template <typename T1, typename T2, typename T3>
95  void CSVPrinter::printRow(T1 val1, T2 val2, T3 val3, int precision) {
96      printValue(val1, false, precision);
97      printValue(val2, false, precision);
98      printValue(val3, true, precision);
99  }
100
101 template <typename T1, typename T2, typename T3, typename T4>
102 void CSVPrinter::printRow(T1 val1, T2 val2, T3 val3, T4 val4, int
        precision) {
```

```
    printValue(val1, false, precision);
    printValue(val2, false, precision);
    printValue(val3, false, precision);
    printValue(val4, true, precision);
}

// Non-template function implementations

void CSVPrinter::begin(long baudRate) {
    Serial.begin(baudRate);
}

void CSVPrinter::printHeader(const char* headers[], int numHeaders) {
    for (int i = 0; i < numHeaders; i++) {
        Serial.print(headers[i]);
        if (i < numHeaders - 1) {
            Serial.print(",");
        }
    }
    Serial.println();
}

void CSVPrinter::printHeader(const char* headerString) {
    Serial.println(headerString);
}

void CSVPrinter::startRow() {
    // This function is for organization/readability - no actual output
}

void CSVPrinter::endRow() {
    Serial.println();
}

#endif // CSV_PRINTER_H
```

Listing 7: CSVPrinter.h

```
# include <Servo.h>
# include "CSVPrinter.h"

// Define variables
int pos = 0;
bool hasRun = false;
Servo myServo;

// Define constants
const int SERVO_PIN = 9; // Pin connected to the servo signal wire
const int LASER_PIN = 3; // Pin connected to the laser
const int LDR_PIN = A0;  // Pin connected to the LDR
const int NUM_SAMPLES = 100; // Number of samples to average for LDR
    reading

/**
 * @brief Holds the mean and standard deviation of LDR readings.
 */
struct LDRStats {
float mean;
```

```
20 float stddev;
21 };
22
23 /**
24  * @brief Averages multiple readings from an LDR connected to the
        specified analog pin.
25  * @param pin The analog pin where the LDR is connected.
26  * @param numSamples The number of samples to average.
27  * @return LDRStats struct containing mean and standard deviation.
28  */
29 LDRStats averageReadLDR(int pin, int numSamples) {
30 float total = 0;
31 float values[20]; // max 20 samples for memory safety
32 if (numSamples > 20) numSamples = 20;
33 for (int i = 0; i < numSamples; i++) {
34     values[i] = analogRead(pin);
35     total += values[i];
36     delay(10); // Small delay to allow ADC to stabilize
37 }
38 float mean = total / (float)numSamples;
39 float sumSq = 0;
40 for (int i = 0; i < numSamples; i++) {
41     sumSq += (values[i] - mean) * (values[i] - mean);
42 }
43 float stddev = sqrt(sumSq / (float)numSamples);
44 LDRStats stats = {mean, stddev};
45 return stats;
46 }
47
48 void setup() {
49     CSVPrinter::begin(9600);
50     myServo.attach(SERVO_PIN);
51     pinMode(LASER_PIN, OUTPUT);
52     digitalWrite(LASER_PIN, HIGH); // Turn on laser
53     myServo.write(0); // Initialize servo position
54     delay(1000); // Wait for servo to reach position
55
56     // Print CSV header
57     CSVPrinter::printHeader("position,ldr_mean,ldr_stddev");
58 }
59
60 void loop() {
61     if (hasRun) {
62         return; // Only run once
63     }
64     for (pos = 0; pos <= 45; pos += 1) {
65         myServo.write(pos);
66         delay(500); // Wait for servo to reach position
67
68         // Take LDR reading
69
70         LDRStats stats = averageReadLDR(LDR_PIN, NUM_SAMPLES);
71         float ldrValue = stats.mean; // Use mean value for CSV
72         float ldrStdDev = stats.stddev; // Use standard deviation for
              CSV
73
74         // Print CSV row
75         // Reverse position to go from 0 to 45
```

```
76          CSVPrinter::printRow(pos, ldrValue, ldrStdDev);
77
78          delay(100);
79      }
80      hasRun = true;
81  }
```

Listing 8: laser_profile_calibration.ino